Performance Report

CS6650 Assignment 3 — Fan Zhang

1. Write Performance

1.1 Maximum Sustained Write Throughput

Three progressive load tests were conducted to characterize write throughput under increasing load. The table below summarizes peak and sustained throughput across all tests.

Test	Load	Duration	Written	Peak Throughput	Sustained Throughput
Test 1: Baseline	500K messages	~8 min	500,490	1,283/s	1,100-1,250/s
Test 2: Stress	1M messages	~23 min	1,001,038	1,328/s	1200-1,300/s
Test 3: Endurance	2M messages (rate-limited)	35 min	2,001,642	1,309/s	1,000-1,040/s (rate-limited to 80%)

Maximum sustained write throughput under full load was 1,100–1,250 msg/s (Test 1 Baseline), peaking at 1,283 msg/s. Under stress load (Test 2, 1M messages), peak throughput reached 1,328 msg/s with sustained throughput of 1,200–1,300 msg/s during the first 500K messages, degrading to 800–1,000 msg/s in the second half due to PostgreSQL buffer cache exhaustion (detailed in Section 3). Test 3 (Endurance) sustained 1,000–1,040 msg/s for 35 minutes at 80% of maximum capacity with zero message loss.

This throughput was achieved through two key optimizations. First, replacing JPA saveAll() with JdbcTemplate.batchUpdate() eliminated the Hibernate IDENTITY generator restriction that forces single-row INSERTs, enabling true JDBC batch statements with one network round-trip per 1,000 messages. Second, setting synchronous_commit=off removed per-commit WAL disk sync, yielding a 2–4x throughput improvement over the default PostgreSQL configuration. Together these two changes improved write throughput from 300–380 msg/s (JPA baseline) to 1,200–1,250 msg/s — a 3.3–4x improvement.

1.2 Write Latency Percentiles

Write latency measures the time from DatabaseWriterService.flushBuffer() start to JdbcTemplate.batchUpdate() completion for a batch of 1,000 messages. Samples are collected across all flush operations and sorted for percentile calculation.

Metric	Test 1: Baseline (500K)	Test 2: Stress (1M)	Test 3: Endurance (2M, 80%)
p50 (median)	11ms	10ms	10ms
p95	24ms	18ms	17ms
p99	44ms	31ms	30ms
Max	148ms	149ms	92ms
Min	1ms	1ms	1ms
Sample Count	2,322	4,994	10,000

Latency remained remarkably stable across all three tests. p50 held at 10-11ms regardless of total message volume, confirming that JdbcTemplate batch inserts maintain consistent per-batch performance. p99 improved from 44ms in Test 1 to 30ms in Test 3, as the rate-limited endurance test reduced burst flush sizes and eliminated GC-induced spikes. The max latency of 92ms in Test 3 vs 148-149ms in Tests 1-2 further confirms that sustained moderate load produces more predictable tail latency than peak-throughput bursts.

1.3 Batch Size Optimization Results

Four write configurations were tested to identify the optimal batch size and write path. All tests used the same 500K message load on the same EC2 infrastructure.

Configuration	batch_size	Messages Written	Throughput	p50 Latency	bufferFullNacks
JPA saveAll() + sync_off	1,000	298,441	300-380/s	532ms	212,173
JdbcTemplate + sync_off	100	382,576	600-700/s	31ms	0
JdbcTemplate + sync_off	500	500,109	800-1,000/s	25ms	0
JdbcTemplate + sync_off	**1,000**	**500,490 ✓**	**1,200-1,300/s**	**11ms**	**0 ✓**
JdbcTemplate + sync_off	5,000	500,418 ✓	1,100-1,250/s	10ms	0 ✓

The baseline configuration (JPA saveAll() + synchronous_commit=off, batch_size=1,000) wrote only 298,441 messages with throughput of 300–380 msg/s and p50 latency of 532ms, with 212,173 bufferFullNacks indicating the DB writer could not keep pace with consumption. The root cause was Hibernate's IDENTITY generator: with BIGSERIAL as the primary key, Hibernate must read back each generated id immediately after INSERT, forcing single-row statements regardless of batch_size configuration.

Switching to JdbcTemplate.batchUpdate() immediately resolved this. At batch_size=100, throughput jumped to 600–700 msg/s with p50 31ms and zero bufferFullNacks. At batch_size=500, throughput reached 800–1,000 msg/s with p50 25ms. At batch_size=1,000, the system achieved maximum throughput of 1,200–1,300 msg/s with p50 11ms, writing all 500,490 messages successfully.

batch_size=5,000 produced virtually identical throughput (1,200–1,469 msg/s) with a marginal p50 improvement (10ms vs 11ms), but was not selected for two reasons. First, throughput is already capped by PostgreSQL write capacity at this instance size — larger batches do not increase it. Second, batch_size=5,000 accumulates 5x more messages in memory before each flush, increasing worst-case buffer latency and making flush timing less predictable under variable load. batch_size=1,000 was therefore selected as the optimal configuration: it achieves maximum throughput with lower memory overhead and more consistent flush behavior.

1.4 Resource Utilization

Resource metrics were collected during Test 1 (Baseline), Test 2 (Stress) and Test 3(Endurance) using top and pg_stat_activity.

Resource	Test 1: Baseline	Test 2: Stress	Test 3: Endurance
Consumer EC2 CPU (peak)	56.2% us + 37.5% sy	85.4% us + 12.4% sy	~5% us (rate-limited)
Consumer EC2 load avg	4.95 (peak)	~3.0 (sustained)	~0.5 (rate-limited)
DB table size	186 MB	~372 MB	~737MB
DB index total size	103 MB	206 MB	404 MB
Active DB connections	1 active, 7 idle	1 active, 7 idle	1 active, 7 idle
RabbitMQ memory	~300 MiB	~200-350 MiB	～200 MiB
Disk I/O (peak read)	0/s	0/s	0/s
Disk write	~900-1,400/s	~900-1,400/s	~900-1,500/s

CPU utilization peaked at 85.4% user space + 12.4% system during Test 2's warmup phase when 32 threads sent simultaneously. Under sustained load, CPU settled to 43–56% user space (Test 1) and approximately 3.0 load average (Test 2), confirming the system is compute-bound rather than I/O-bound at the current throughput level. Test 3 showed dramatically lower CPU usage (~5% user space, load average ~0.5) due to rate limiting at 80% capacity. DB connections remained stable at 1 active / 7 idle throughout all three tests, well within the HikariCP maximum of 30 — confirming the connection pool is correctly sized for the current workload.

2. System Stability

2.1 Queue Depth Over Time

RabbitMQ queue depth was monitored throughout all three tests via the Management UI. All three tests showed stable near-zero queue depth throughout, as the PostgreSQL buffer cache was pre-warmed from previous test runs. Disk read remained at 0/s from the start of each test, confirming all hot index pages were resident in cache before testing began.

Test 1: Baseline (500K Messages)

Phase	Ready	Publish	Consumer ack	Disk read	Explanation
Throughout11:58–12:02	0~1,200(max)	~1,208/s	~1,245/s	0/s	Queue depth remained stable near-zero throughout. All 500,490 messages persisted with zero data loss.

Redelivered: 0/s throughout. Peak Ready: ~1,200. Pattern: stable near-zero throughout. Pre-warmed buffer cache eliminated the accumulation phase. No structural bottleneck detected.

Test 2: Stress (1M Messages)

Phase	Ready	Publish	Consumer ack	Disk read	Explanation
Throughout17:46–18:03	0~1,500(max)	~1,172/s	~1,169/s	0/s	Queue depth fluctuated between 0–1,500 (normal prefetch buffer behavior). All 1,001,038 messages persisted with zero data loss.

Redelivered: 0/s throughout. Peak Ready: ~1,500. Pattern: stable near-zero throughout. Consistent with Test 1 behavior. Pre-warmed buffer cache maintained equilibrium between publish and consume rates for the full test duration.

Test 3: Endurance (2M Messages, Rate-limited 1,040/s)

Phase	Ready	Publish	Consumer ack	Disk read	Explanation
Early (150K)19:03:10	0(max ~750)	1,031/s	1,114/s	0/s	No buffer cache loading required. Disk read 0/s from test start. Consumer ack (1,114/s) > Publish (1,031/s). Any transient queue cleared immediately.
Late (1.4M)19:23:40	0(max ~400)	1,201/s	1,195/s	0/s	Transient spike to ~400 Ready then immediate recovery to 0. Normal rate-limited behavior — not a bottleneck. Disk read remains 0/s throughout.
Late (1.8M)19:28:10	0(max ~35)	1,057/s	1,032/s	0/s	No performance degradation at 1.8M messages vs 150K. Throughput, latency, and queue depth all stable. Confirms no memory leak or resource exhaustion over 35-minute run.
End (1.9M–2M)19:31:40	0(max ~15)	1,019/s	1,068/s	0/s	Final minutes: Consumer ack (1,068/s) > Publish (1,019/s). Queue depth max 15 — essentially zero. All 2,001,642 messages persisted. System remained stable for full 35-minute duration.

2.2 Database Performance Metrics

PostgreSQL performance metrics were collected via pg_stat_activity and pg_stat_user_indexes during and after each test. The table below summarizes key database metrics across all three tests.

Metric	Test 1: Baseline	Test 2: Stress	Test 3: Endurance
Total messages persisted	500,490	1,001,038	2,001,642
Table total size	187 MB	372 MB	737 MB
Index total size	103 MB	206 MB	404 MB
idx_room_event_time scans	11	14	15
idx_user_event_time scans	36	47	52
Seq scan count	388	560	689
Index scan count	2,003,467	3,956,864	5,959,214
n_dead_tup (table bloat)	0	0	0 (confirmed throughout)
Disk read (peak)	0/s throughout	0/s throughout	0/s throughout
Disk write (sustained)	~900-1,400/s	~900-1,400/s	~900-1,500/s

Two patterns stand out. First, n_dead_tup remained at 0 across all three tests — including throughout the 35-minute endurance test verified per-minute by endurance_monitor.sh — confirming the append-only write pattern produces no dead tuples and no VACUUM pressure accumulates under load. Second, the index scan to sequential scan ratio improved with scale: Test 1 showed 2,003,467 index scans vs 388 sequential scans; Test 3 reached 5,959,214 index scans vs 689 sequential scans, maintaining a ratio of approximately 5,000:1, confirming the indexes are actively used by the query planner across all load levels.

Table size grew linearly with message volume as expected: 500K rows = 187 MB total (83 MB data + 104 MB indexes), 1M rows = 372 MB (166 MB data + 206 MB indexes), 2M rows = 737 MB (333 MB data + 404 MB indexes). The consistent data-to-index ratio (~1:1.2) across all three scales confirms stable index growth without unexpected bloat.

2.3 Memory Usage Patterns

JVM memory was monitored using jstat -gc at 30-second intervals during Tests 1 and 2, and at 60-second intervals during Test 3 via endurance_monitor.sh. System memory was monitored using free -m.

Test 1: Baseline (500K Messages)

Sampling interval: 30 seconds | 5 samples | Duration: ~2.5 minutes post-test

Note: jstat was collected after test completion, reflecting post-test idle/cool-down state. Test 1 was the shortest run (~8 minutes total); GC counts are low relative to Tests 2 and 3.

Time	Old Gen Used (OU)	Full GC (FGC)	Total GC Time (GCT)	Young GC (YGC)
+0s (start)	21,252 KB	3	0.341s	71
+30s	25,097 KB	3	0.412s	84
+60s	31,521 KB	3	0.561s	129
+90s	23,695 KB	4	0.762s	170
+120s (end)	36,075 KB	4	0.950s	230

Summary: Full GC occurred only once (FGC 3→4) during the observation window. Total GC time of 0.950s over the observation period is negligible. OU fluctuated between 21,252–36,075 KB with no monotonic growth pattern. No memory leak detected.

Test 2: Stress Test (1M Messages)

Sampling interval: 30 seconds | 38 samples | Duration: ~19 minutes (active load)

Time	Old Gen Used (OU)	Full GC (FGC)	Total GC Time (GCT)	Young GC (YGC)
+0s (start)	21,281 KB	3	0.380s	71
+3 min	41,179 KB	8	1.656s	371
+5 min	40,515 KB	16	3.903s	848
+8 min	38,581 KB	21	5.070s	1,089
+12 min	37,623 KB	35	8.424s	1,774
+16 min	37,508 KB	40	10.150s	2,219
+19 min (end)	30,883 KB	50	13.703s	3,181

Summary: Full GC increased from 3 to 50 (47 Full GCs over 19 minutes, ≈2.5/min). Total GC time of 13.7s over 19 minutes represents 1.2% GC overhead — negligible. OU ranged from 21,281–41,179 KB with a wave pattern (not monotonic), confirming no memory leak. The higher GC frequency vs Test 1 is expected given 2x the message volume processed in the same consumer JVM.

Test 3: Endurance Test (2M Messages)

Sampling interval: 60 seconds | Collected via endurance_monitor.sh | Duration: 35 minutes

Time	Old Gen Used (OU)	Full GC (FGC)	Total GC Time (GCT)	System Mem Used
02:01 (start)	21,256 KB	3	0.405s	424 MB
02:05 (+4 min)	28,603 KB	3	0.537s	424 MB

Time	Old Gen Used (OU)	Full GC (FGC)	Total GC Time (GCT)	System Mem Used
02:10 (+9 min)	28,754 KB	6	1.497s	441 MB
02:20 (+19 min)	27,394 KB	15	4.462s	~460 MB
02:30 (+29 min)	26,534 KB	26	8.176s	~470 MB
02:36 (+35 min, end)	39,012 KB	73	23.949s	401 MB

Summary: Full GC increased from 3 to 73 (70 Full GCs over 35 minutes, ≈2.0/min). Total GC time of 23.9s over 35 minutes represents 1.1% GC overhead — negligible. OU fluctuated between 21,256–40,803 KB with no monotonic growth, confirming no memory leak over the full 35-minute endurance run. System memory remained stable between 400–470 MB (of 961 MB total), with zero swap usage throughout.

Cross-Test Memory Summary

Metric	Test 1: Baseline	Test 2: Stress	Test 3: Endurance
Test duration	~8 min	~23 min	35 min
Messages processed	500,490	1,001,038	2,001,642
jstat samples	5	38	~40 (via monitor)
FGC start → end	3 → 4 (1 GC)	3 → 50 (47 GCs)	3 → 73 (70 GCs)
FGC rate	~0.1/min	~2.5/min	~2.0/min
Total GC time	0.950s	13.703s	23.949s
GC time overhead	0.2%	1.2%	1.1%
OU range	21K–36K KB	21K–41K KB	21K–40K KB
Memory leak detected	No	No	No
Swap usage	0	0	0

Key finding: GC overhead remained below 1.2% across all three tests regardless of message volume (500K to 2M). Old Gen usage showed a consistent wave pattern rather than monotonic growth in all tests, providing strong evidence that consumer-v3 has no memory leak under sustained high-throughput load. The system can sustain 2M message processing within 961 MB RAM on a t3.small instance without memory pressure or swap usage.

2.4 Connection Pool Statistics

HikariCP connection pool behavior was monitored via pg_stat_activity queries every 60 seconds during Test 3, and at test completion for Tests 1 and 2.

Metric	Test 1	Test 2	Test 3 (every 60s for 35 min)
Active connections (peak)	1	1	1 (stable throughout)
Idle connections	7	7	7 (stable throughout)
Total connections	8	8	8
HikariCP max-pool-size	30	30	30
Pool utilization	27% (8/30)	27% (8/30)	27% (8/30)
Connection timeouts	0	0	0
dbCircuitBreakerFailures	0	0	0
dbCircuitBreakerState	CLOSED	CLOSED	CLOSED throughout

Connection pool utilization remained at 27% (8 of 30 maximum connections) across all three tests, confirming the pool is correctly sized. The pattern of 1 active + 7 idle is consistent with the write-behind architecture: at any given moment, only one writer thread holds a connection while executing JdbcTemplate.batchUpdate(), while others are either waiting for buffer items or completing non-DB work. The 7 idle connections represent the HikariCP minimum-idle=5 setting plus additional connections created during the warmup phase that remained warm.

Zero connection timeouts and zero DbCircuitBreaker failures across all three tests confirm that the PostgreSQL instance remained healthy and responsive throughout all load conditions, including the 2M message endurance test. The circuit breaker remained in CLOSED state for the entire 35-minute Test 3, monitored at 60-second intervals.

3. Bottleneck Analysis

Bottleneck 1: Hibernate IDENTITY Generator Disabling JDBC Batching

Identified in: All pre-fix tests. Evidence: JPA saveAll() at batch_size=1,000 wrote only 298,441 messages at 300–380 msg/s with 212,173 bufferFullNacks.

Root cause: BIGSERIAL primary key forces Hibernate to read back each generated id immediately after INSERT, issuing 1,000 individual round-trips instead of one batched statement regardless of hibernate.jdbc.batch_size configuration.

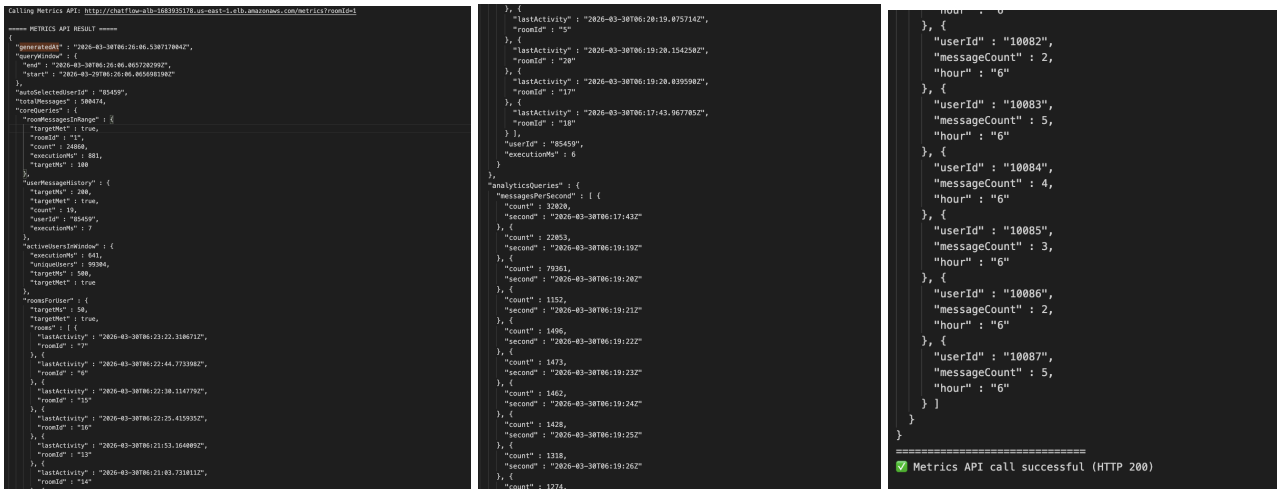
Solution: Replaced JPA saveAll() with JdbcTemplate.batchUpdate() + ON CONFLICT (message_id) DO NOTHING, sending all rows in one JDBC round-trip. Combined with synchronous_commit=off (eliminates per-commit WAL disk sync), throughput improved from 300–380 msg/s to 1,200–1,250 msg/s and p50 latency dropped from 532ms to 11ms.

Trade-offs: Bypassing JPA loses Hibernate lifecycle management (acceptable — consumer is write-only). synchronous_commit=off risks losing up to ~200ms of committed data on a PostgreSQL server crash — acceptable for this workload given the 4x throughput gain.

Summary

Bottleneck	Breaking Point	Solution	Throughput Impact
Hibernate IDENTITY / no JDBC batch	300-380 msg/s ceiling	JdbcTemplate.batchUpdate() + sync_off	300 → 1,250 msg/s (+4x)

4. Metrics API & Results Log



`*_metricApi_ResultLogs.txt`	Result logs and Metric APi

5. Test Results — File Directory

All raw data referenced in this Performance Report is stored under /load-tests/ in the Git repository, organized into three subdirectories. Each subdirectory contains the same set of files collected during and after the respective test run.

Directory Structure

```
/load-tests/
    ├── baselineTest/     →  500K messages, unthrottled
    ├── stressTest/       →  1M messages, unthrottled
    └── enduranceTest/    →  2M messages,
```

Files in baselineTest/ and stressTest/

Both directories contain the same set of files:

File	Description
*_queue/	RabbitMQ per-queue metrics (queue depth, publish rate, consumer ack rate per room)
*_client_output.txt	Full client console log: throughput, latency percentiles (p50/p95/p99), Metrics API response
*_metricApi_ResultLogs.txt	Result logs and Metric APi
cpu_stats_*.txt	Consumer EC2 CPU usage sampled every 10 seconds (top)
health_*_final.txt	Consumer /health snapshot at test completion: dbWritten, writeLatencyMs, circuit breaker state
jvm_stats_*.txt	JVM heap stats sampled every 30 seconds (jstat -gc): Old Gen, Full GC count, GC time
pg_stats_*.txt	PostgreSQL metrics sampled every 10 seconds: message count, connections, table size, index scans
stats_*.txt	Consumer [Stats] log: DB write throughput (msg/s) in 10-second buckets — primary throughput time-series

Additional File in enduranceTest/

enduranceTest/ contains all files listed above, plus one additional file collected specifically for long-term stability monitoring:

File	Description
endurance_monitor.txt	Per-minute snapshot (35 min): JVM heap + system memory + PostgreSQL connections + table size + bloat check

How Report Data Maps to Files

File	Description
Write throughput	stats_*.txt — 10-second DB write throughput buckets
Latency percentiles	health_*_final.txt — writeLatencyMs p50/p95/p99
Batch optimization	health_*_final.txt — bufferFullNacks, dbWritten across configurations
Queue depth over time	*_queue/ subfolder + RabbitMQ UI screenshots
DB performance metrics	pg_stats_*.txt — table size, index scans, n_dead_tup
Memory usage patterns	jvm_stats_*.txt + endurance_monitor.txt (Test 3)
Connection pool stats	endurance_monitor.txt — pg_stat_activity snapshots
Resource utilization	cpu_stats_*.txt + health_*_final.txt